Exercise 5

Github Link

Stage A — Today's system

Read `PostgresSink` in `plant.py`, then think through this timeline:

- gateway 1 inserts rows that get the ids 100–114 and commits 100 ms later
- gateway 2 inserts rows 115–129 and commits at once.
- A row gets its `id` when it is inserted, but others see it only when it is committed.

Are the missing readings in the table? Why did the detector never see them?

The Detector uses `WHERE ID > ${ID}`, with `${ID}` representing the highest seen ID
This means that the Detector ignores asynchronous writes to the DB
e.g.: ID's might go: 1 2 *10* `3 4 5 6 7 8 9`
Since 10 is the highest it saw, all subsequent writes won't be caught by the `WHERE` clause

---

Start the detector again, then the full plant, and let both run for ~60 s. Watch
the detector's lines while the plant runs:
Stop the detector right after the
plant: afterwards it only works through its backlog.

what happens to `delay`, `backlog` and `missing`, and why?

Delay

Delay grows, because there are 60 machienes (e.g.: $\frac{600}{min}$), but only $\frac{\sim 333}{\text{sec}}$ ($\frac{1}{3ms}$) can be consumed.
e.g.: The Backlog grows

Backlog

Backlog grows because of previously mentioned reasons.

Missing

Missing increases because of reasons mentioned in the first question.

Stage B — Decide

	Requirement
R1	Scale: 600 readings/s in the lab; the design must grow to 100,000 readings/s.
R2	Complete and ordered: the model gets *every* reading of a machine, in `seq` order (the model counts `missing` and `out-of-order` readings).
R3	Fresh: anomalies at most 5 s (p95) after the reading, at full rate.
R4	Independent consumers: the raw readings also feed a training-data archive and a live dashboard. Adding a consumer must not slow down, change or break the others, and must not add load to PostgreSQL.

Compare these two options against R1–R4 (add your own idea if you have one):

1. keep PostgreSQL, but poll it better (e.g. several pollers, each for a part of the machines);
2. the gateways write every reading into Kafka (or another log-based platform) instead of into PostgreSQL.

For each option: which requirements does it meet, which are at risk? Then decide, and estimate for your choice: how many detector instances does the model (3 ms per reading) need for 600 readings/s, and for 100,000? A small office that logs one temperature reading per minute is perfectly served by option 1: where, roughly, is the tipping point?

Write it down as a short decision record (half a page: options, decision, reasons, numbers). PostgreSQL stays the system of record for the machine registry in any case.

Requirements Breakdown

Requirement	PostgreSQL with improved polling	Kafka
R1 - Scale	Can handle the lab with parallel detectors; 100,000 readings/s risks database contention and high polling overhead.	Partitions support parallel processing; brokers and detectors would have to be sized appropriately (e.g.: Autoscaling)
R2 - Complete and ordered	Possible, but partitioning pollers alone does not fix skipped commits (would require reliable tracking and ordering per machine, which would be hard to coordinate)	Use `machine_id` as the key. e.g.: machine sends data to the same partition every time. → ordered reliable producer settings (e.g.: Wait for confirmation; resend; no duplicated; etc...) Recovery must also preserve or rebuild model state. (e.g.: WAL)
R3 - Fresh	Possible at low load. However, polling intervals and backlogs for processing threaten the 5 s target IF they get too big	Possible with sufficient processing capacity and balanced partitions; Kafka alone does not magically make a system super fast tho.
R4 - Independent consumers	Fails as proposed: each additional polling consumer adds PostgreSQL load.	Separate consumer groups provide independent positions and processing, without reading raw data from PostgreSQL. Shared broker capacity must remain sufficient.

Decision: Kafka for raw readings, while PostgreSQL remains for the machine registry. Kafka better supports the required scale (especially with parallelism), ordered processing per machine, replay and independent archive/dashboard consumers. It is also the obvious choice for futureproofing (e.g.: What if more machines are added, or another data source for all machienes)

Estimation

Each detector processes at most:

$[1/0.003 \approx 333 \text{ readings/s}]$

Input rate	Theoretical instances
600 readings/s	2
100,000 readings/s	300

The theoretical figures exclude overhead and unforseen influences.
→ e.g.: 300 instances at full rate leave no spare capacity.
For Kafka, at least as many partitions as active detectors would be appropriate.
→ verify latency under load.

Tipping point: There is no universal rate imo. For this model, one detector reaches its theoretical limit around 333 readings/s. PostgreSQL polling remains reasonable for infrequent readings and simple consumer needs, but for sustained, fresh traffic, the parallel processing, replay and multiple independent consumers of Kafka are the obvious choice